Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/1531743.1531753

RESEARCH-ARTICLE

A multi-streaming SIMD architecture for multimedia applications

JIHCHING CHIU, National Sun Yat-Sen University, Kaohsiung, Taiwan

YULIANG CHOU, National Sun Yat-Sen University, Kaohsiung, Taiwan

HUAYI TZENG, National Sun Yat-Sen University, Kaohsiung, Taiwan

Open Access Support provided by:

National Sun Yat-Sen University

PDF Download
1531743.1531753.pdf
04 February 2026
Total Citations: 7
Total Downloads: 393

Published: 18 May 2009

Citation in BibTeX format

CF '09: Computing Frontiers Conference
May 18 - 20, 2009
Ischia, Italy

Conference Sponsors:
SIGMICRO

CF '09: Proceedings of the 6th ACM conference on Computing frontiers (May 2009)
https://doi.org/10.1145/1531743.1531753
ISBN: 9781605584133

A Multi-streaming SIMD Architecture for Multimedia Applications

Jih-Ching Chiu
Department of Electrical Engineering, National Sun Yat-sen University, Kaohsiung, 804, Taiwan
chiujihc@ee.nsysu.edu.tw

Yu-Liang Chou
Department of Electrical Engineering, National Sun Yat-sen University, Kaohsiung, 804, Taiwan
d943010010@student.nsysu.edu.tw

Hua-Yi Tzeng
Department of Electrical Engineering, National Sun Yat-sen University, Kaohsiung, 804, Taiwan
c669975@hotmail.com

ABSTRACT
Current MMX-like extensions provide a mechanism for general–purpose processors to meet the growing performance demand of multimedia applications. However, the computing performance of these extensions is often limited because they only operate on a single data stream. To overcome this obstacle, this paper presents an architecture named "multi-streaming SIMD architecture" that enables one SIMD instruction to simultaneously manipulate multiple data streams. The proposed architecture is a Processor-In-Memory-like register-file architecture including SIMD operating logics for general-purposed processors to further extend current MMX-like extensions to obtain high performance. To efficiently and flexibly realize the proposed architecture, an operation cell is designed by fusing the logic gates and the storage cells together. The operation cells then are used to compose a register file with the ability of performing SIMD operations called "Multimedia Operation Storage Unit (MOSU)". Further, many MOSUs are used to compose a multi-streaming SIMD computing engine that can simultaneously manipulate multiple data streams and exploit the subword parallelisms of the elements in each data stream. Three instruction modes (global, coupling, and isolated modes) are defined for the MMX-like extensions to modulate the amount of parallel data streams and to efficiently utilize the computation resources. Simulation results show that when the multi-streaming SIMD architecture has four 4-register MOSUs, it provides a factor of 3.3x to 5.5x performance improvement compared with Intel's MMX extensions on eleven multimedia kernels.

Categories and Subject Descriptors
C.1.2 [**Processor Architectures**]: Multiple Data Stream Architectures

General Terms
Performance, Design, Measurement, Experimentation

Keywords: SIMD, PIM, Processor-in-memory, Multimedia extensions, MMX, Streaming Processing, Streaming Computing

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
CF'09, May 18–20, 2009, Ischia, Italy.
Copyright 2009 ACM 978-1-60558-413-3/09/05...$5.00.

1. INTRODUCTION
Current computer applications are multimediarich, including audio and video compression, 2D image processing, 3D graphics, signal processing etc. How to meet the computational requirements of these applications is a challenge of modern processor design. The most popular solution in the past decade has been the enhancement of the general-purposed processor with multimedia extensions such as Intel's MMX [1], Sun VIS for SPARC [2], and PowerPC's ALtiVec [3]. The key idea in these extensions is the exploitation of subword parallelism [4] in a single instruction multiple data (SIMD) fashion. It has been shown that these extensions can improve the performance of many multimedia kernels and applications. Nevertheless, their limitations can be summarized as follows:

- One instruction manipulates limited-length (64 or 128-bit) data, and hence it lacks the ability to exploit the full data level parallelism (DLP) in multimedia applications.
- MMX-like extensions are restricted by the unit-stride constraint; that is, all the elements of the vector must be consecutively arranged in memory. Hence, increasing the width of the multimedia register is not able to load so many arranged subword elements as the size of the multimedia register, thus achieving diminishing returns.
- The overhead for data conversion and reorganization.

The rest of the paper is organized as follows: In Section 2, we describe the related works. In Section 3, we describe the structure and functionality of the multi-streaming SIMD architecture at the (assembly) programming level. In Section 4, a possible datapath for the multi-streaming SIMD architecture are sketched. Section 5 presents the performance results to evaluate the potential performance gains achievable by the proposed architecture. The paper is summarized in Section 6.

2. RELATED WORKS
Many recent works notice and deal with these limitations. A method for improving performance is to exploit more instruction level parallelism (ILP) among the SIMD instructions. However, Talla et al. [5] revealed that adding more SIMD units and increasing the issue width did not impact performance positively. Slingerland and Smith [6] propose a SIMD architecture that implicitly unpacks while loading, implicitly packs while storing, and provides stride load and store instructions. The Matrix Oriented Multimedia (MOM) extension [7] contains instructions that can be viewed as vector versions of SIMD instruction and it allows an arbitrary stride between consecutive rows but requires a

unit stride between consecutive row elements. The Matrix Register File (MRF) [8] allows loading the data stored consecutively in memory to a column of the register file, where a column corresponds to corresponding subwords of different registers. In complex streamed instruction (CSI) set [9], a single instruction can process a data stream of arbitrary length and stride, and combines complex memory accesses, program control for vector sectioning, and complex computations on multiple data in a single operation. The current researches above focus on providing a single seamless data stream for a single SIMD unit. In contrast to these current researches, we propose a multi-streaming SIMD architecture, which enables one SIMD instruction to manipulate multiple data sets in parallel, is designed trying to exploit the advantages of both stream processing [10] and MMX-like short-vector extensions.

The Streaming processing is a computer programming paradigm that allows a series of operations (kernel functions) to be applied to each element in a regular data set (stream) in parallel. The key feature of streaming processing is that streaming processing decouples computation and memory accesses by accessing the data sets before the computation and updating the computed data to memory. Recently there have been many stream processors to be presented such as Imagine [11], Merrimac [12], Cell BE [13], and current GPGPUs [14]. They are suitable for data-centric applications which are compute-intensive with mass data parallelism but not for general-proposed processing.

Another kind of performance-improving methods for multimedia applications is the Processor-In-memory (PIM) concept, which capitalizes on merging the processor unit with its memory unit on the same chip. Several architectures based on this concept have been proposed such as VIRAM [15], FlexRAM [16], Active Pages [17], DCS [18], etc. The most immediate benefit provided by these architectures is the increased on chip bandwidth and low memory latency. Compared to the multimedia-extension extended processor, these PIM systems can meet the much higher levels of performance required by multimedia applications. However, they have two major limitations. First, each PIM system is fixed in its applications, which limits its adaptability, scalability and cost effectiveness. Second, a pre-processing stage is required to partition applications between a processor and an intelligent memory system, which introduces software overhead.

3. THE MULTI-STREAMING SIMD ARCHITECTURE

3.1 The Multi-streaming SIMD Design Philosophies

As mentioned previously, the main reason why current MMX-like extensions lack the ability to exploit the full DLP of the multimedia applications is that these MMX-like extensions are single data stream SIMD operations and can only exploit subword parallelism. In order to overcome these obstacles and obtain high performance, we propose a multi-streaming SIMD architecture, which further extended current MMX-like extensions to support multiple data streams, is designed trying to exploit the advantages of both stream processing and MMX-like short-vector extensions. Instead of increasing the number of SIMD units and the issue width to exploit more ILP, and instead of simply widening the multimedia registers to exploit more DLP, the multi-streaming SIMD architecture in this paper organizes multiple SIMD units and registers into a multi-streaming SIMD computing engine. This architecture enables one SIMD instruction to simultaneously manipulate multiple data sets. Under the control of a SIMD instruction, many SIMD units perform the same operations in parallel on different data sets. Each SIMD unit also exploits the subword parallelisms of the elements in each data set.

In Figure 1, a comparison between the two different paradigms (MMX-like and Multi-streaming SIMD paradigms) is applied to a very simple code example, which adds all 16-bit elements of two two-dimensional arrays together. The MMX-like operations only vectorize the inner loop j through the exploitation of subword parallelism. On the other hand, the multi-streaming SIMD operations not only implicitly unroll the outer loop i through the four SIMD units but also vectorize the inner loop j through the exploitation of subword parallelism at the same time. By this way, the computation ability of proposed architecture is limited only by the resource limitations and bus bandwidth.

Moreover, we also propose the hierarchical implementation methods based on the Processor-In-Memory (PIM) concept to easily and flexibly realize the multi-streaming SIMD architecture. In contrast with building large multiported register files to provide the necessary bandwidth for the SIMD units, we fuse SIMD operations within the register file to allow aggressive streamed

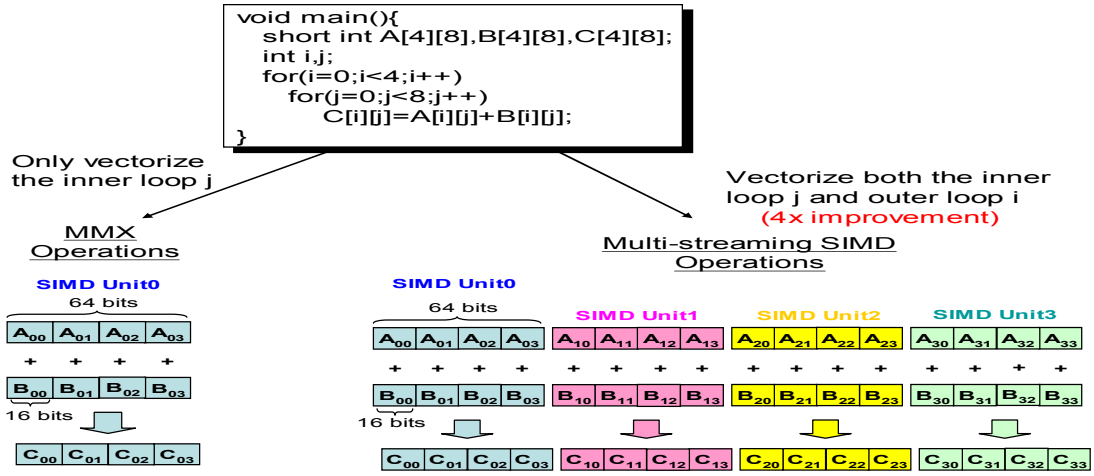

Figure 1: A comparison between the MMX and Multi-streaming SIMD paradigms.

multimedia operations. By partitioning the register file across the SIMD unit inputs and integrating operations and storages together, the SIMD units can be scaled efficiently to sustain the high arithmetic rates demanded by multimedia application and many excellent two-dimensional data manipulation instructions can be realized efficiently, by which multiple SIMD processing elements stored in the proposed architecture can be re-permuted, transposed and accumulated under the controls of one instruction. The detail implementation methods will be described in Section 4 in this paper. In software aspects, three instruction modes (global, coupling, and isolated modes) are defined for the MMX-like extensions to modulate the amount of parallel data streams and to efficiently utilize the computation resources. Programmers can dynamically configure the SIMD operation resources in the instruction level to match the various data-computing styles in different number of vector streams. Current MMX-like extensions can easily be further extended to the multi-streaming SIMD by the following two steps. First, designers make their multimedia extension ISA support the three operation modes described in Section 3.2 and the space addressing mode described in Section 3.3. Second, they should increase the number of the SIMD unit according to the implementation methods described in Section 4. In this paper, the Intel MMX technology is chosen as the baseline single-streaming SIMD extensions and further be extend to multi-streaming SIMD.

3.2 Three Instruction Modes for Dynamically Configuring the SIMD Computing

A major feature of the multi-streaming SIMD architecture is its three-mode instruction set (global, coupling, and isolated modes). Current MMX-like instruction set can be upgraded by prefixing "G", "C" and "I" to the MMX-like instructions to become the three-mode instruction set. Given *N* SIMD units and *M* registers, where *M* is divisible by *N*, the isolated mode can operate on *N* different data streams. However, programmers can only use *M*/*N* logical registers to program. Comparing to the isolated mode, the coupling mode operates on fewer data streams (only *N*/2 data streams), whereas programmers can use more logical registers (increased to *2M*/*N* logical registers) to program. This mechanism provides programmers with greater flexibility. It is possible to run a smaller number of data streams, each of which requires more registers, or a larger number of data streams, each of which requires fewer registers. In contrast to the isolated and coupling modes, the global mode only operates on one data stream. In this mode, however, programmers can use all the *M* registers. In the proposed architecture, the main purposes of the global mode instruction are (1) to rapidly transfer and manipulate data in different data streams and (2) to maintain compatibility with original MMX-like instruction set.

In Figure 2, given four SIMD units and eight 64-bit registers, an example of all the programmer-visible SIMD operation resources and data streams supported by the three instruction modes are shown. The global mode instruction (Figure 2(a)) allows the multi-streaming SIMD architecture to act as a conventional single-streaming SIMD unit. Only one SIMD unit is active, and programmers can use all the eight registers. If there is no prefix prior to the MMX-like instructions, these instructions are

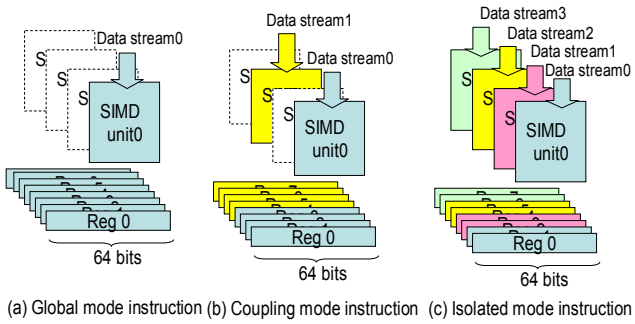

Figure 2: Relationship between three-mode instruction and resource allocation.

identified as the global mode instructions. Therefore, the application programmed in original MMX-like instructions can still run well in proposed architecture. Figure 2(b) shows that in the coupling mode, the multi-streaming SIMD architecture acts as two conventional single-streaming SIMD units performing identical operations on two different data streams. Therefore, the coupling mode exploits double the DLP of the global mode. However, programmers can only use 4 logical registers. Reg0~3 and Reg4~7 are allocated to the SIMD unit0 and unit2 respectively. The actions of two SIMD units are symmetric. For example, when the coupling mode MOVQ instruction, "CMOVQ Reg0, Reg3", is executed, SIMD unit0 moves the contents of Reg3 to Reg0. At the same time, SIMD unit2 moves the contents of Reg7 to Reg4. Similarly, "CMOVQ Reg7, Reg4" leads to the same actions. However, "CMOVQ Reg0, Reg7" is an invalid coupling mode instruction format. In the coupling mode, using the registers belonging to other SIMD units is forbidden. Programmers should use the global mode instruction to transfer or manipulate the data of different data streams among registers. The isolated mode is analogous to the coupling mode. The isolated mode has the highest DLP (four data streams can be simultaneously handled in this example), but there are only 2 logical registers for programmers to use.

3.3 The Space Addressing Mode for Accessing Multiple Data Streams

In order to provide different data streams for the SIMD units, the space addressing mode is proposed in the multi-streaming SIMD architecture. The data streams are specified by two stream control registers: the Start Address Register (SAR), which indicates the start address of the first data stream and the Space Offset Register (SOR), which indicates the space between two data streams. As Table 1 shows, we take the architecture with four SIMD units and eight 64-bit registers as an example to describe the relationship between space addressing mode and three instruction modes. The italicized letter *i* in the "Example instructions" column indicates the destination register of the example instruction, and *k* in the "Meaning" column indicates the actual registers into which the data will be loaded. For example, "CMOVQ mm3, [SAR, SOR]" is the couple mode instruction with space addressing. It moves the data addressed by [SAR] to mm3 and the data addressed by [SAR+SOR] to mm7. Moreover, programmers can set the SOR as 8 and use "IMOVQ mmi, [SAR, SOR]" to access a successive 256-bit data. In this situation, four SIMD units appear as a 256-bit

wide SIMD engine. By the space addressing, multiple successive data streams can be accessed for multiple SIMD units. The space addressing can also cooperate with the strided or gather/scatter addressing [19] to access multiple non-successive data streams but with regular strides in the memory.

3.4 An Example of Using the Three-mode Instructions

In this subsection, an example code is presented to demonstrate how to effectively use the three-mode instructions to implement the RGB to YUV color conversion [20], which is an important kernel in multimedia codecs such as JPEG and MPEG. Because all MMX instructions are integer operations, the equation for RGB to YUV color conversion is given as below.

$$\begin{bmatrix} Y \\ U \\ V \end{bmatrix} = \frac{1}{32768}\begin{bmatrix} 9798 & 19235 & 3736 \\ -4784 & -9437 & 4221 \\ 20218 & -16941 & -3277 \end{bmatrix}\begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

Assuming that the workload is sixteen pixels of interleaved RGB, and assuming that the proposed architecture has four SIMD units and eight 64-bit registers, Figure 3 shows the kernel steps used to transform RGB to YUV and the execution results of the mm0, mm2, mm4, and mm6 after each step. In this example, each SIMD unit deals with its own data set. Each of the SIMD unit converts four successive pixels of interleaved RGB to YUV with 15-bit accuracy; that is, Unit0, Unit1, Unit2, and Unit3 handle pixel 0~3, pixel 4~7, pixel 8~11, and pixel 12~15, respectively. The *RGB*, *Zero*, and *c_RGB* parameters of the initial step presented in Figure 3(a) represent the start addresses of the sixteen pixels of RGB, the 64-bit local variable containing zero value, and the 64-bit local variable containing the four 16-bit coefficient values $C_RC_BC_GC_R$, respectively.

In the initial step, three move instructions prepare the start addresses of local variables and data streams. The first instruction in Step 1 sets up the space between two data streams. Since each SIMD unit is responsible for four successive pixels (4x3=12 bytes), the SOR is set at 12. The second instruction in Step 1 is an isolated mode instruction that loads four different data streams into four SIMD units using the space addressing mode. Step 2 unpacks the 8-bit R, G, and B values to increase the data size to 16 bits. This instruction accesses the *Zero* local variable simply via the register indirect addressing mode, and broadcasts the value of *Zero* local variable to four SIMD units for unpacking. Step 3 loads four 16-bit coefficient values $C_RC_BC_GC_R$ also via the register indirect addressing mode and broadcasts them to the four SIMD units. These values are then multiplied by the corresponding RBGR values. As in Step 2, Step 3 also uses the register indirect addressing mode.

Effective use of MMX instructions requires that the data be positioned in the registers to take advantage of SIMD capabilities of MMX technology. Therefore Step 4 uses a new instruction, "MTRANS", which performs a 4x4 matrix transposition in one cycle, to rearrange the data in the registers to facilitate the operation of MMX extensions. After Step 4, the global mode instructions can be used to sum up the multiplied partial results, and therefore the mm6 contains the calculated Y values of pixel 0, 4, 8, and 12. The partial results of Y values of pixel 1, 5, 9, and 13 are also calculated in the mm0. To avoid destroying the calculated partial results in the mm0, the six following steps are similar to those of Figure 3(a) but work on the other set of registers (mm1, mm3, mm5, and mm7). In these six following steps, the SAR0 in the initial step should have the value of *RGB+4*, and the SAR1

Table 1: The examples of space addressing mode in three instruction modes.

Mode	Example instructions	Meaning
Global	GMOVQ mm*i*,[SAR,SOR]	mm*i*← Mem[SAR+ SOR]

Coupling	CMOVQ mm*i*,[SAR,SOR]	For j=0 to 1{
		k= (*i*+4j)mod 8
		mm*k*←Mem[SAR+j*SOR]
		}

Isolated	IMOVQ mm*i*,[SAR,SOR]	For j=0 to 3{
		k=(i+2j)mod 8
		mm*k*←Mem[SAR+j*SOR]
		}

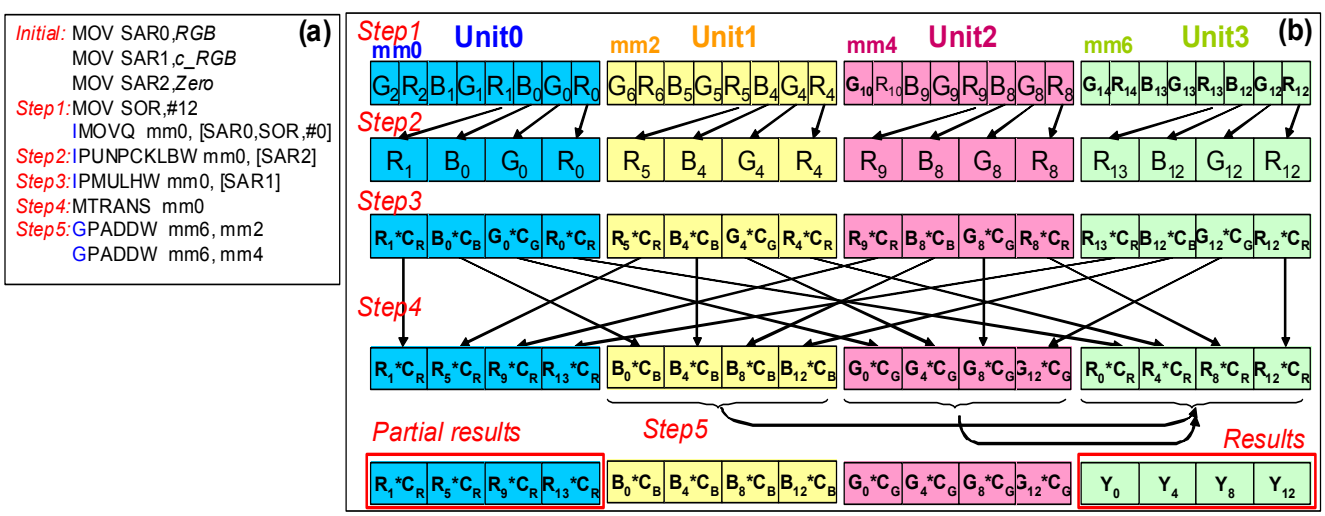

Figure 3: Kernel steps of transforming RGB to YUV and the execution results of mm0, mm2, mm4, and mm6 after each step.

should be the address of another local variable containing the four 16-bit coefficient values $C_G C_R C_B C_G$. Repeating the six steps three times produces all sixteen Y values. The U and V values can also be obtained in a similar way, but the values of the coefficients C_R, C_G, and C_B are different for the calculations of U and V.

3.5 High-level Programming Language Support

Since our instruction set is based on current MMX-like extensions, and then we further extend these MMX-like instructions to support three operation modes, these new extensions can become the new intrinsics of these MMX-like extensions. Using the new intrinsics facilitates programmers to build the function libraries to support programming in high-level languages. This can free programmers from managing registers and assembly programming. Further, the stream programming languages like CUDA [21] and Brook [22] can be further extended to support our architecture because our architecture is based on the stream computing concept.

4. IMPLEMENTATION

4.1 Hardware Design Concepts of the Multi-streaming SIMD Architecture

Figure 4 shows a simplified block diagram of the 3-stage pipeline processor with the multi-streaming SIMD architecture. The multi-streaming SIMD architecture can act two different roles, first simple as the register file while general-purposed instructions are being executed and second as the SIMD units support multiple data streams while SIMD instructions are being performed. Since the single central register file does not scale well with large numbers of SIMD units, we base on the Stream/SIMD/DRF register organization [23], which employs a stream buffer to prefetch data streams and capture data locality and partitions the local registers across the ALU inputs to reduce the cost of register files, to realize the proposed architecture. And we further fusing the computing logics and the register storage together by the following implementation steps:

1. To fuse computing logics and D flip-flops together as an operation cell. Hence, the operation cell has the dual capabilities of operation and storage for bit slicing (see the upper right block of Figure 4).
2. To implement a special SIMD unit named MOSU (Multimedia Operation Storage Unit), which has the dual capabilities of subword SIMD operations and storage, by means of combining many operation cells (see the right block of Figure 4).
3. Four MOSUs are bound to construct a basic multi-streaming SIMD computing cell named BMMOC (Basic Multi-streaming Multimedia Operation Cell) to handle different data streams (one, two, or four data streams) according to the three-mode instructions. (see the lower right block of Figure 4)
4. Every Four BMMOCs are grouped with a SB (switch box) and four grouped BMMOCs are also grouped with a SB. The SB is responsible for handling the data communications among the BMMOCs. Through the connections of SBs, many BMMOCs are bound to construct a multi-streaming SIMD computing engine. (see the center block of Figure 4)

By this hierarchical implementation method, the hardware designers can provide different levels of SIMD computational ability for the general-purposed processors according to their needs. Further, all the data for Multimedia operations will be placed at the multi-streaming SIMD computing engine and be handled in parallel. The numbers of read and write ports of register file do not need to be increased for general-purposed operations.

The main difference between the stream processor and the proposed architecture involves data communication between the local registers. The communication ability of the stream processor's local registers is limited. Therefore, the stream processor is suited for applications performing many operations on each element of a long, 1D stream. In the proposed architecture, fusing the operations and the register file together means that many excellent stream-interconnection operations that are useful when applications are not fully data parallel, can be realized efficiently for multimedia computations. Examples of these operations include the intra-register data manipulation, the inter-register data manipulation and even the two-dimensional data manipulation (perform both intra-register and inter-register manipulation in one instruction) of SIMD data elements. Moreover, the stream processors and the PIM systems mentioned in Section 2 are coprocessors. The scalar and vector hardware of these architectures are implemented separately and communicate only through the coprocessor interface; therefore, vector instructions can not directly access the register files in the scalar core. The coprocessor instructions are the only way to move scalar data to the register file in the vector hardware for vector instructions to use. On the other hand, the proposed architecture is a functional unit in a general-purposed processor. The computation results of the SIMD instructions can be directly accessed through the general-purposed instructions without requiring data movements.

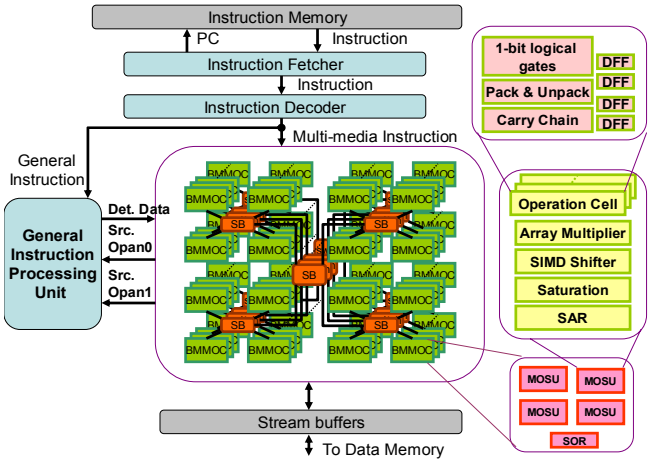

Figure 4: A simplified block diagram of the three-stage pipeline architecture with the multi-streaming SIMD architecture.

4.2 Basic Components of the Multi-streaming SIMD Architecture

As illustrated in Figure 4, the multi-streaming SIMD architecture comprises groups of BMMOCs. The BMMOC is the basic component handling different data streams (one, two, or four data streams) according to the three-mode instructions. Each BMMOC contains four MOSUs, which has the ability to perform all the MMX operations and keep the results in its own registers, and a Space Offset Register (SOR). The MOSU, in turn, contains an OP cell array, a SIMD shifter, a tree array multiplier, a saturation unit, and a Start Address Register (SAR). It can perform the 64-bit SIMD operations in the single data stream, such as the addition, subtraction, packed, unpacked and logical operations, and it also has four 64-bit registers to store data for those operations. Hardware designers can adjust the amount of DFFs according to their needs. The operation cell is the smallest component of the proposed architecture and is able to perform bit-level arithmetic and logical operations. An operation cell can be divided into two parts: (1) Execution logics. (2) Storage cells. The execution logics consist of the 1-bit logical gates (AND, OR, and XOR) used for logical operations, a carry chain for constructing a Carry-Lookahead Adder (CLA), and the pack & unpack logics for the pack & unpack operations. Another part is the storage cells which includes four 1-bit DFFs. The operation results can be latched in one of the 4 DFFs. Besides, designers can adjust the amount of DFFs according to their needs.

Figure 5 shows the prototype of the operation cell, each operation cell has three outputs (Operand_A, Operand_B, and Output_C). The Operand_A is connected to the input of multiplier, shifter, pack & unpack logics of operation cells, and basic execution logics in the same operation cell. The data of the Operand_A can only be from one of the four DFFs. The Operand_B is different from the Operand_A. The data of the Operand_B can be from the storage cells, the registers of other MOSUs, or the memory, and it is connected to another input of the multiplier, pack & unpack logics of operation cells, and basic execution logics in the same operation cell. The Operand_A and B can also provide two operands for the general instruction processing unit while the general-purposed instruction is being executed. Finally, the operation results needed to be saturated can be sent to the saturation unit through the Output_C. The values which will be written into the storage cells are from different sources (the multiplier, saturation unit, SIMD shifter, pack & unpack logics, basic execution logics, and general-purpose processor). According to the operations of the instructions, the operation cell selects the correct value to be stored into the correct DFF. In order to avoid the inefficiency of the multi-streaming SIMD computing engine as a general-purpose register file, the Input_A is designed. The write back values of the general-purposed processor can be directly written into the storage cells from the Input_A without the delays of the execution logics.

5. PERFORMANCE EVALUATION

5.1 Experimental Setups

5.1.1 Benchmarks

The eleven benchmark kernels are chosen to analyze the characteristics of the proposed architecture, which are *idct*, *motion1*, *motion2*, *rgb2ycc*, *h2v2upsample*, *saturate*, *addblock*,

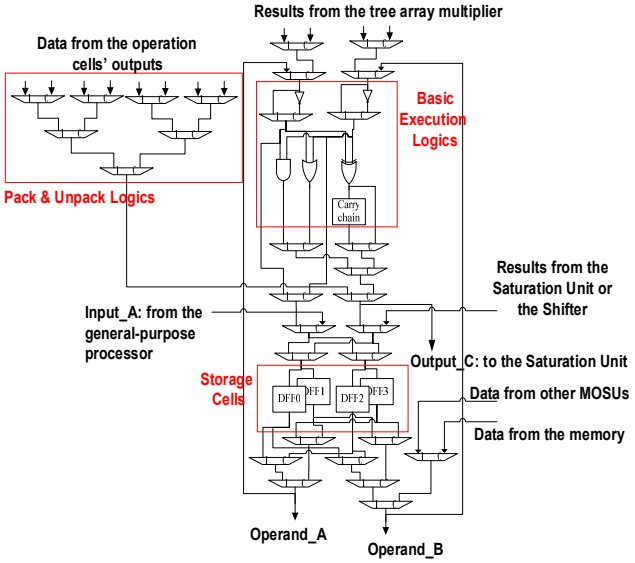

Figure 5: The prototype of the operation cell.

quantization, *zigzag*, *MCT* and *bilinear interpolation*. All of them are representative kernels of video and image processing applications. Nine kernels are from JPEG, MPEG, and H.264, and two kernels are from the face detection and image resampling techniques. The *idct* kernel performs a 2D inverse discrete cosines transform over 8x8 matrices. The *motion1* performs a sum of absolute differences between two 16x16 image matrices to perform a motion estimation operation while *motion2* performs a sum of quadratic differences. The *rgb2ycc* kernel transforms a RGB image into YCC format and the *h2v2upsample* performs a 2x2 zoom over the whole image for the JPEG decoder. The *saturate* kernel performs the saturation of 16-bit elements to 12-bit range in MPEG decoder and *addblock* performs the MPEG frame reconstruction. The *quantization* and *zigzag* are basic kernels of H.264. Finally, the *MCT* (Modified Census Transform [24]) is one of major approaches for the face detection task and the *bilinear interpolation* [25] is one of the basic resampling techniques in computer vision and image processing.

5.1.1.1 Methodology

As mentioned in Section 3.1, we choose Intel's MMX extensions as the baseline single-streaming SIMD extensions and further extended them into multi-streaming SIMD. The extended MMX extensions are named the multi-streaming MMX in the following paragraphs of this section. The MMX [26] has a multimedia register file with eight 64-bit registers and all the MMX instructions are 2-address instructions. Two other single-streaming multimedia extensions, SSE4 [27] and iwMMXt [28], are also considered in the simulations. The register width (128 bits) of the SSE4 is the double of that (64 bits) of the MMX. It can be the representative of increasing the register size to obtain performance in the simulations. The iwMMXt is an extension to the Intel's XScale ARM processors and its functions are similar to those of the MMX extensions. It contains sixteen registers of 64-bits and it is a 3-address instruction set. The iwMMXt is chosen in the simulation to observe whether the increase of register amount and 3-address instruction set can improve the performance. The simulation goal is to observe the performance enhancement of

different architectures (SSE4 has wider register width than MMX, the iwMMXt is a 3-address instruction set and it has more registers than MMX, and the multi-streaming MMX can explore more DLP through the multi-streaming SIMD architecture), so the simulations only consider the integer operations of these multimedia extensions, and this is why the SSE extensions [29], whose register size and amount are the same as MMX ,is not considered in the simulations.

The simulations will be used to evaluate the achievable performance gains among the multi-streaming MMX, SSE4, and iwMMXt architectures. Those results are compared with the results of the MMX architecture as the base. The implementations of the benchmarks are based on vender supplied application notes [30]-[32]. The base processor is a single-issue single-cycle processor and then is enhanced with the SSE4, iwMMXt, MMX, and multi-streaming MMX respectively. Regarding the processor enhanced with the multi-streaming MMX, we will focus on the performance boost of one BMMOC, which is basic element of the multi-streaming architecture can handle multiple data streams according to the three instruction modes.

The processors enhanced with the SSE4, iwMMXt, MMX, and multi-streaming MMX are implemented in Verilog and simulated on ModelSim by feeding the benchmarks into them to obtain the simulation results. And we assume that there is a perfect stream buffer that can promptly provide data for these processors and enable the load/store instruction to be finished in one cycle. The execution cycles of the instructions of these multimedia extensions are all according to the vender supplied documentations [31][32].

5.2 Performance Analysis

5.2.1.1 Simulation1: the Performance Enhancement under Different BMMOC Hardware Resources

The goal of this simulation is to show how many registers in a MOSU are appropriate for programmers to flexibly and efficiently code their programs. Figure 6(a) shows the speedups attained by the three multi-streaming MMX-enhanced processors with different BMMOC hardware resources while compared with the MMX-enhanced processor. The x-axis represents the eleven benchmark programs, and the y-axis represents the speedup (the total execution cycles of the MMX-enhanced processor divided by the total execution cycles of the multi-streaming MMX-enhanced processor). The BMMOC(8 Regs), BMMOC(16 Regs), and BMMOC(32 Regs) represent the simulation results of the multi-streaming MMX-enhanced processors with one BMMOC, which includes four MOSUs, and the register amounts of each MOSU are 2,4, and 8 respectively.

In the BMMOC(32 Regs) simulations, the multi-streaming MMX-enhanced processor has 4x computation resources than the MMX-enhanced processor, so the benchmarks written in traditional MMX instructions are easy to be modified to support multiple data streams. Simply prefixing "I" to the MMX instructions in the benchmarks and modifying the load/store instructions, which provide data to the SIMD computation unit, to support the space addressing mode. Because four MOSUs of BMMOC perform the identical operations but they work on four different data streams, the multi-streaming MMX-enhanced processor can provide a speedup over the MMX-enhanced processor of up to 4x.

In the BMMOC(16 Regs) simulations, the multi-streaming MMX-enhanced processor has three additional SIMD computation unit and double amounts of MMX registers compared to the MMX-enhanced processor, and therefore 4 logical registers provided for programmers to program the benchmarks in isolated mode instructions. Since 4 logical registers are not enough to capture the short-term data locality of some benchmarks, the intermediate results need to be stored back to memory and then the speedups decrease. Similar situation appears in BMMOC(8 Regs) simulation.

To raise the arithmetic intensity (the ratio of arithmetic to memory reference), the "MTRANS" instruction is considered in the simulation. The "MTRANS" can transpose a 4x4 matrix whose four rows are stored in four MOSUs respectively. Moreover, it can be easily and efficiently realized in the proposed architecture by the proposed implementation method. Figure 6(b) shows the simulation results of the multi-streaming MMX-enhanced processor supporting the "MTRANS" instruction, and it demonstrates that this new instruction can benefit the *idct*, *rgb2ycc*, and *MCT* kernels. The 2D idct algorithm takes a 2D array of data and operates first on rows of data, then on columns

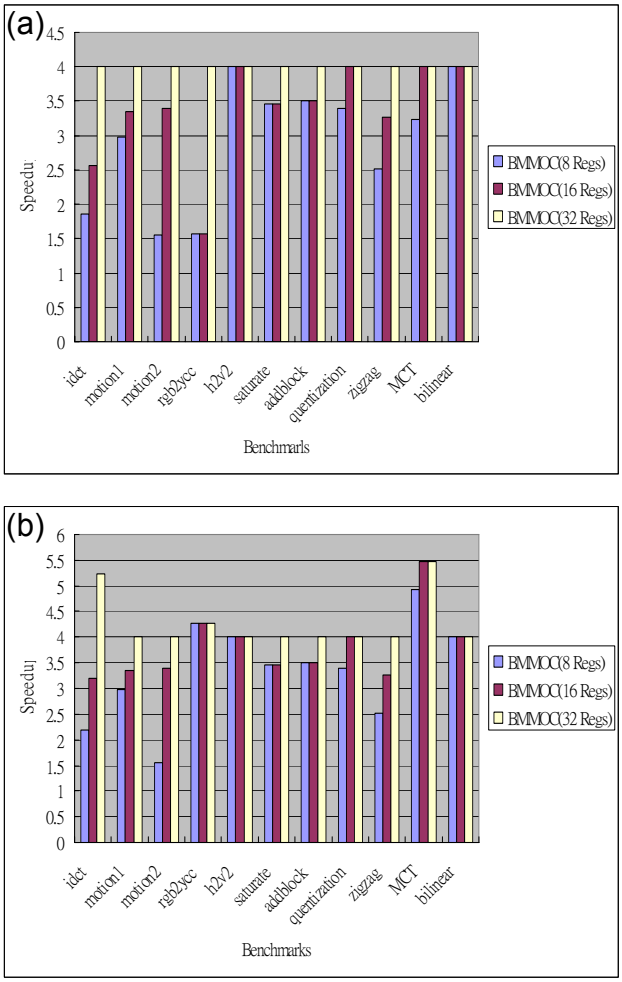

Figure 6: (a) Speedups of different BMMOC hardware resources over MMX. (b) Speedups of different BMMOC hardware resources supporting the "MTRANS" instruction over MMX

of data. An array is usually laid out in memory one row after another. MMX technology is ideal for manipulating rows in this type of organization, as row elements reside in subsequent addresses. On the other hand, this organization doesn't work for operating on columns. The "MTRANS" instruction makes rows into columns and enables MMX technology to be able to execute the same operations on all elements of the column in parallel. Therefore, the speedup of the *idct* can be enhanced and the reason for the enhancement of *rgb2ycc* and *MCT* kernels is similar. With the "MTRANS" instruction support, the BMMOC (32 Regs) simulation obtains performance gains ranging from 4x to 5.5x over the traditional MMX-enhanced architecture and the BMMOC (16 Regs) simulation can also well outperform MMX for all the kernels (up to 3.3x~5.5x speedups).

In conclusion, the registers of BMMOC(8 Regs) are too few to capture the data locality of the benchmarks. Therefore, its performance enhancement is limited. On the contrary, the BMMOC(32 Regs) can provide high performance and low software reengineering overheads. Considering the cost-effectiveness, the BMMOC(16 Regs) supporting "MTRANS" instruction is an appropriate choice.

5.2.1.2 Simulation2: Speedups of SSE4, iwMMXt, and BMMOC(16 Regs) over MMX

The goal of this simulation is to observe the performance enhancement of different architecture, we chose the simulation results of the BMMOC(16 Regs) supporting "MTRANS" instruction to compare with the MMX, SSE4, and iwMMXt. Figure 7 shows the speedups attained by the SSE4-enhanced, iwMMXt-enhanced processors when compared with the MMX-enhanced processor.

In the iwMMXt simulation, the new integer instructions, the "PMINSW" instruction (Minimum of Packed Signed Word Integers) and "PMAXSW" instruction (Maximum of Packed Signed Word Integers), make it outperform MMX for the *motion1*, *saturate*, and *addblock* kernels. Since the iwMMXt is a three-address instruction set, the iwMMXt can slightly outperform MMX for the *idct*, *motion2*, *rgb2ycc*, *quantization*, *zigzag*, and *MCT* kernels. The SSE4 has the 2x register width (128 bits) then MMX, so the SSE4 shows speedups over MMX about 2x for most of the kernels. Similar to the iwMMXt simulation, the "PMINSW", "PMAXSW", and another new instruction, the "MPSADBW" (Multiple Packed Sums of Absolute Difference) instruction that performs eight 4-byte wide sums of absolute difference operations per instruction, make the SSE4 can well outperform MMX for the *motion1*, *saturate* and *addblock* kernels. Comparing the BMMOC(16 Regs) with the MMX, SSE4, and iwMMXt, BMMOC(16 Regs) outperforms for most of the kernels. There are three cases (*motion1*, *saturate* and *addblock*) where BMMOC(16 Regs) is not much more effective than SSE4. This is because SSE4 support the new instructions mentioned above, however BMMOC(16 Regs) only supports MMX instructions and needs more instructions to accomplish the same operations.

In conclusion, enlarging the register width can improve performance, but it is revealed that the performance will achieve diminishing returns while the width is being further increased [7]. The 3-address instructions make the programmers have more flexibility to program. Because the destination register can not be the same as one of the source registers that is helpful to reserve

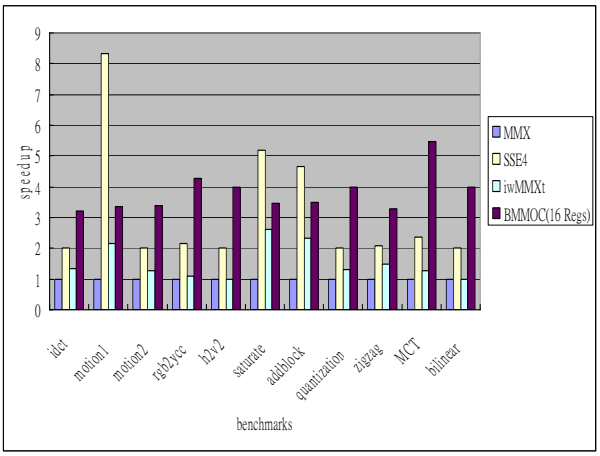

Figure 7: Speedups of SSE4, iwMMXt, and BMMOC(16 Regs) over MMX

the constant coefficients or intermediate results, the performance can be slightly improved. Simply increasing the number of register can not enhance the performance. It needs additional software engineering efforts, such as loop-unrolling and software pipeline techniques etc., to improve performance and utilize the registers. New instructions can only benefit some specific multimedia kernels. Different from designing specific instructions to speedup specific multimedia kernels, the proposed architecture can comprehensively enhance the multimedia kernels.

5.3 Hardware Complexity Analyses

We synthesized the basic components of the proposed architecture, the MMX-enhanced processor, and the multi-streaming MMX-enhanced processor by the Synopsys Design Compiler in a 0.13 um static CMOS technology. The results are shown in Table 2. The critical path of the MOSU starts from the source register, passes through the array multiplier and Carry-Lookahead adder, and ends in the destination register. The critical path of the BMMOC is also dominated by the multiply-accumulate instruction, but it has another 6 ns delay at the multiplexer that manages the data flowing between the four MOSUs. The synthesis results of BMMOC(16 Regs) and BMMOC(16 Regs) supporting "MTRANS" instruction are almost the same. The hardware cost and performance degradation of extending BMMOC to support "MTRANS" instruction are low. The area spent on implementing the multi-streaming MMX enhanced processor with BMMOC (16 Regs) supporting "MTRANS" instruction is 2.8 times the area spent on implementing the MMX-enhanced processor, which has one SIMD operation unit, thirty-two 32-bit general-purpose registers, and extra eight 64-bit MMX registers. The area spent on implementing the multi-streaming MMX-enhanced processor with BMMOC (32 Regs) is 3.1 times area spent on implementing the MMX-enhanced processor.

6. CONCLUSION

In this paper, a multi-streaming SIMD architecture is proposed to further extend current MMX-like extensions to manipulate multiple data sets in parallel. The contributions of this paper are described as below:

Table 2: Hardware complexity of each component

Hardware	Total area(um^2)	Timing (ns)
SB	**337690**	**5.25**
MOSU(4 Regs.) **MOSU(8 Regs.)**	**1981010** **2216864**	**45.62** **47.35**
BMMOC(16Regs.) **BMMOC(32Regs.)** **BMMOC(16Regs.)+*MTRANS***	**8043789** **9089142** **8055147**	**51.62** **53.24** **52.12**
The MMX enhanced processor	**3178951**	**48.87**
The multi-streaming MMX-enhanced processor with BMMOC(16 Regs)+*MTRANS*	**8928507**	**55.47**
The multi-streaming MMX-enhanced processor with BMMOC(32 Regs)	**9873853**	**56.07**

- We proposed a Processor-In-Memory-like register-file architecture including SIMD logic for general-purposed processors to further extend current MMX-like extensions to obtain high performance. Under the controls of one MMX-like instruction, many SIMD units perform the same operations in parallel on different data sets and each SIMD unit also exploit the subword parallelisms of the elements in each data set. Therefore, the major restriction (lack the ability to exploit more DLP) of current MMX-like extensions can be resolved.
- Three instruction modes (global, coupling, and isolated modes) are defined for the MMX-like extensions to modulate the amount of parallel data streams and to efficiently utilize the computation resources.
- The operation cell, which fuses logic gates and D flip-flops together, is proposed to hierarchically implement the proposed architecture. Based on this hierarchical implementation method, hardware designers can provide different levels of SIMD computational ability for general-purpose processors
- The two-dimensional data manipulation concept by which multiple SIMD processing elements stored in the proposed architecture can be re-permuted, transposed and accumulated under the controls of one instruction.

The Intel's MMX technology is chosen as the baseline single-streaming SIMD extensions to be further extended them into the multi-streaming SIMD architecture. The eleven representative kernels of current multimedia applications are chosen to evaluate the performances. The area spent on implementing the multi-streaming SIMD architecture with a BMMOC (16 registers) supporting "MTRANS" instruction is 2.8 times the area spent on implementing the MMX-enhanced processor, and the speedups of proposed architecture can over MMX technology ranging from 3.3x to 5.5x.

How to promptly obtain data from memory and efficiently transmit data among BMMOCs will be the two major issues while scaling the size of the multi-streaming SIMD architecture. In the future, we will focus on designing the suitable communication networks among the BMMOCs and data stream buffers for large size multi-streaming SIMD architecture. We will also provide an extended stream programming paradigm for programmers to facilitate the programming on our proposed architecture. By the plenty of registers and computation resources, our proposed architecture can used to construct the large amount of registers in general-proposed processors to support multi-streaming SIMD operations and make the processors have the best abilities to handle data-intensive multimedia applications in real time.

7. ACKNOWLEDGMENTS

We would like to acknowledge all the members of the microprocessors and microprocessor system development laboratory at National Sun Yat-sen University. We would also thank the anonymous reviewers for their valuable comments and suggestions.

8. REFERENCES

[1] Peleg and U. Weiser, "MMX technology extension to the Intel architecture," IEEE Micro, vol.16, no. 4, Aug.1996, pp. 42-50.

[2] M. Tremblay, J. M. O'Connor, V. Narayanan, and L. He, "VIS speeds new media processing," IEEE Micro, vol. 16, no. 4, Aug. 1996, pp. 10-20.

[3] K. Diefendorff, P.K. Dubey, R. Hochsprung, and H. Scales, "AltiVec Extension to PowerPC Accelerates Media Processing," IEEE Micro, vol. 20, no. 2, pp. 85-95, Mar/Apr. 2000.

[4] R.B. Lee, "Subword parallelism with MAX-2," IEEE Micro, vol. 16, no. 4, pp. 51-59, Aug. 1996.

[5] D. Talla, L.K. John and D. Burger, "Bottlenecks in Multimedia Processing with SIMD Style Extensions and Architectural Enhancements", IEEE Transactions on Computers, vol. 52, No. 8, Aug 2003 ,pp.1015-1031

[6] N. Slingerland and A. Smith, "Measuring the performance of multimedia instruction Sets," IEEE Transactions on Computers, vol. 51, Nov. 2002, pp. 1317–1332.

[7] J. Corbal, M. Valero, and R. Espasa, "Exploiting a new level of DLP in multimedia applications," in Proc. Int. Symp. Microarchitecture, 1999, pp. 72–81.

[8] A. Shahbahrami, B. Juurlink, and S. Vassiliadis, "Accelerating Color Space Conversion Using Extended Subwords and the Matrix Register File," in IEEE International Symposium on Multimedia, Dec. 2006, pp. 37 – 46.

[9] D. Cheresiz, B. Juurlink, S. Vassiliadis, and H.A.G. Wijshoff, "The CSI multimedia architecture," IEEE Transactions on Very Large Scale Integration Systems, vol. 1, Jan. 2005, pp. 1-13.

[10] AMD Corp.,"AMD Stream Computing, " Sep. 2008. DOI=http://ati.amd.com/technology/streamcomputing/sdkdwnld.html

[11] B. Khailany, W.J. Dally, U.J. Kapasi, P. Mattson, J. Namkoong, J.D. Owens, B. Towles, A. Chang, S. Rixner,"Imagine: media processing with streams," IEEE Micro, vol. 21, no. 2, pp. 35-46, Mar/Apr. 2001.

[12] W.J. Dally, F. Labonte, A. Das, P. Hanrahan, Ahn Jung-Ho, J. Gummaraju, M. Erez, N. Jayasena, I. Buck,T.J. Knight, U.J. Kapasi, "Merrimac: Supercomputing with Streams," in Proc. ACM/IEEE Supercomputing Conference, Nov. 2003, pp. 35 – 35.

[13] M. Gschwind, H.P. Hofstee, B. Flachs, M. Hopkins, Y. Watanabe, and T. Yamazaki, "Synergistic Processing in Cell's Multicore Architecture," IEEE Micro, vol. 26, no. 2, pp. 10-24, Mar/Apr. 2006.

[14] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, "NVIDIA Tesla: A Unified Graphics and Computing Architecture," IEEE Micro, vol. 28, no. 2, pp. 39-55, Mar/Apr. 2008.

[15] D. Patterson and C. Kozyrakis, "Scalable Vector Processors for Embedded Systems," IEEE Micro, vol. 23, no. 6, pp. 36-45, Nov.-Dec. 2003.

[16] Y. Kang, M. Huang, S. Yoo, Z. Ge, D. Keen, V. Lam, P. Pattnaik, and J. Torrellas, "FlexRAM: Toward an Advanced Intelligent Memory System," in Proc. IEEE Int'l Conf. Computer Design, 1999.

[17] M. Oskin, F.T. Chong, and T. Sherwood, "Active Pages: A Computation Model for Intelligent Memory," in Proc. 25th Ann. Int'l Symp. Computer Architecture, pp. 192-203, June 1998.

[18] J. Y. Kang, S. Gupta, and J. L. Gaudiot, "An Efficient Data-Distribution Mechanism in a Processor-In-Memory (PIM) Architecture Applied to Motion Estimation," IEEE Trans. on Computers, vol. 57, no. 3, pp.375-388, Mar. 2008.

[19] P.T. Hulina, L.D. Coraor, L. Kurian, and E. John, "Design and VLSI Implementation of an Address Generation Coprocessor," in IEE Proc. Computers and Digital Techniques, vol. 142, no. 2, pp. 145-151, Mar.1995.

[20] Intel Corp,"Using MMX Instructions to Convert RGB to YUV Color Conversion", MMX? Technology Manuals and Application Notes. DOI=http://softwarecommunity.intel.com/articles/eng/1713.htm

[21] NVIDIA Corp,"CUDA Programming Guide 1.1," 2007. DOI=http://developer.download.nvidia.com/compute/cuda/1_0/NVIDIA_CUDA_Programming_Guide_1.0.pdf

[22] I. Buck, "Brook Specification v0.2,"2003. DOI=http://merrimac.stanford.edu/brook/

[23] S. Rixner, W.J. Dally, B. Khailany, P. Mattson, U.J. Kapasi, and J.D. Owens, "Register organization for media processing," in Sixth International Symposium on High-Performance Computer Architecture, Jan.2000, pp. 375 – 386.

[24] B. Froba, and A. Ernst, "Face detection with the modified census transform," in IEEE International Conference on Automatic Face and Gesture Recognition, May. 2004, pp. 91 – 96.

[25] Ze-Nian Li and Mark S. Drew, Fundamentals of Multimedia, Prentice-Hall, 2004.

[26] A. Peleg, and U. Weiser, "MMX technology extension to the Intel architecture," IEEE Micro, vol. 16, no. 4, Aug. 1996, pp. 42-50.

[27] Intel Corp," Intel SSE4 programming Reference," May 2007. DOI=http://www.intel.com

[28] N.C. Paver, M.H. Khan, and B.C. Aldrich, "Accelerating mobile multimedia using Intel Wireless MMX technology," in IEEE International Symposium on Multimedia Software Engineering, Dec. 2004, pp. 491 –498.

[29] S. Thakkar and T. Huff, "The internet streaming SIMD extensions," Intel Technol. J., May 1999, pp. 26–34.

[30] Intel Corp, MMX? Technology Manuals and Application Notes.DOI=http://softwarecommunity.intel.com/articles/eng/1713.htm

[31] Intel Corp, "Intel XScale(R) Core Developer's Manual," DOI=http://www.intel.com/design/intelxscale/273473.htm

[32] Intel Corp," Intel 64 and IA32 Architecture Optimization Reference manual," November 2007. DOI=http://www.intel.com